Laurent Schroeder

Language-Guided Drone Control

Semester Thesis

Robotics and Perception Group
University of Zurich

Supervision

Ismail Geles
Ángel Romero
Dr. Elie Aljalbout
Prof. Dr. Davide Scaramuzza

June 2025

Contents

Abstract **iii**

Nomenclature **v**

1 Introduction **1**
1.1 Related Work 2

2 Methodology **3**
2.1 Data Collection Pipeline 3
2.2 Models 4
2.2.1 MLP 4
2.2.2 LangBeT 5
2.2.3 Personal Transformer 7
2.3 Overall Pipeline Overview 7

3 Experiments **10**
3.1 MLP 10
3.1.1 Simulation 10
3.1.2 Real World 12
3.2 LangBeT 14
3.3 Personal Transformer 15

4 Discussion **19**
4.1 Conclusion 19
4.2 Future Work 20

Abstract

In this work, we present a flexible pipeline for language-conditioned drone control, combining real-time data collection in a ROS environment with vision-language models and neural network policies. We designed a system capable of capturing synchronized state, velocity commands, and images which are then transformed into embeddings using CLIP or ResNet models. Using this pipeline, we trained both a baseline MLP and a custom Transformer model to map observations to drone actions. The MLP showed robustness to slight linguistic variations, while the Transformer demonstrated temporal reasoning and achieved a higher success rate in simulation, 75% vs. 72.22%. Despite challenges with more complex architectures like LangBeT, our results suggest that transformer-based policies hold great potential for scaling to various tasks and environments. This work provides a foundation for future research into language-guided robotic control in real-world environments with imitation learning.

Nomenclature

Acronyms and Abbreviations

RPG	Robotics and Perception Group
LLM	Large Language Model
VLM	Vision Language Model
UAV	Unmanned Aerial Vehicle
VLN	Visual Language Navigation
MLP	Multi Layer Perception
FoV	Field of View
BeT	Behavioral Transformer
LangBeT	Language Behavioral Transformer
MSE	Mean Squared Error

Chapter 1

Introduction

UAVs are increasingly being used in both indoor and outdoor environments to perform a wide range of tasks, including aerial surveillance, package delivery [6], and search and rescue missions [9]. These applications require UAVs to operate in complex, dynamic, and often unknown environments, while interacting with both static and dynamic objects, as well as with humans.

Unlike legged or wheeled robots, UAVs operate in fully three-dimensional space, which introduces additional challenges for control and navigation. Their flight is sensitive to disturbances such as wind and limited visibility, and their typically high speeds demand fast and reliable reactions to avoid collisions with both moving and stationary obstacles.

Achieving robust UAV autonomy thus requires reliable perception, accurate motion planning, and safe interaction with humans and other agents. While early systems relied on GPS signals and pre-defined trajectories, the emergence of modern computer vision techniques has opened the door to more flexible solutions. In particular, VLN for UAVs is a promising area of research, where the goal is to control drones through natural language commands conditioned on images. Ideally, a UAV should be able to interpret a high-level instruction such as *"fly through the gate"* and convert it into low-level control commands using onboard cameras. However, this remains a challenging problem, especially in settings with only a single forward-facing camera, due to limited FoV, partial observability, and uncertain environmental dynamics.

In recent years, advances in LLMs and VLMs have enabled significant progress in grounded language understanding and have driven development in VLN. Yet, most of these advancements have been limited to legged or wheeled robots navigating on flat, human-made terrains [2, 5]. In contrast, the use of VLN for UAVs has received relatively little attention [7], [1].

This project addresses this gap. We present a system for language-guided drone control, which maps high-level natural language instructions to low-level velocity commands by conditioning on the current and four previous image and state observations. The model is trained using imitation learning from expert demonstrations.

Our main contributions are:

1. We implemented a flexible data collection pipeline compatible with both simulation, for example the Flightmare simulator [10], and real-world environments.
2. We demonstrate the effectiveness of our setup in a real-world setting using a small MLP model for language-conditioned control.
3. We show that our pipeline can be extended to transformer-based models capable of capturing temporal dependencies.

1.1 Related Work

Early research on language-conditioned control has primarily focused on ground-based robots, with notable contributions such as NaVILA [2] and LeLaN [5].

NaVILA [2] introduces a two-level architecture that separates high-level vision-language understanding from low-level locomotion control. A vision-language model processes images and generates mid-level actions in natural language, which are then translated into precise motor commands by a separate locomotion policy. While this design shows strong generalization across different ground robot platforms, it has not been extended to aerial systems.
Similarly, LeLaN [5] leverages foundation models to label in-the-wild video data with natural language instructions, enabling the training of state-of-the-art policies for zero-shot, language-conditioned object navigation. However, similar to NaVILA, LeLaN's evaluations are limited to legged and wheeled robots operating in structured indoor and outdoor environments.

More recently, attention has turned toward language-conditioned UAV control, though this remains an emerging research area. FlightGPT [1] combines supervised fine-tuning with reinforcement learning to train UAVs to follow natural language instructions but only in simulation.
Additionally, UAV-VLN [7] extends VLN methods to UAVs by leveraging multimodal inputs to generate navigational commands that generalize to unseen real-world environments. Despite its strengths, UAV-VLN does not incorporate temporal navigation history, which can be critical for complex tasks, and its path planning is not always optimal.

While these systems illustrate significant progress in vision-language understanding for UAVs, they often suffer from limitations such as lack of flexibility for fine-tuning or requiring extensive training data. In contrast, our work provides a modular and practical framework that enables easy collection of task-specific training data, followed by fine-tuning and evaluation using a relatively small dataset. This makes our system more adaptable to application-specific UAV tasks.

Chapter 2

Methodology

In this chapter, we first describe our data recording pipeline. We then detail the models used to learn control policies from this data: a MLP as a baseline, the conditional BeT (LangBeT) model for goal-conditioned action prediction, and a custom Transformer architecture that captures temporal dependencies in sequential input. Finally, we summarize how these components integrate into a unified control pipeline, as illustrated in Figure 2.6.

2.1 Data Collection Pipeline

To enable a flexible and reusable pipeline applicable across different tasks and environments, we designed our data collection framework to separately store image data and metadata. For each trajectory, the current image is saved in a folder named `Images{timestamp}`, while the corresponding state and velocity command are logged in a .csv file named `data{timestamp}.csv`.

The quadrotor's state is represented as a 7-dimensional vector:

$$[\text{pos_}x, \text{pos_}y, \text{pos_}z, \text{ori_}x, \text{ori_}y, \text{ori_}z, \text{ori_}w]$$

where the position is given in cartesian coordinates and the orientation is encoded as a quaternion. The velocity command is a 4-dimensional vector:

$$[v_x, v_y, v_z, \text{yaw_rate}]$$

Both the image folder and the .csv file are stored inside a parent folder named after the corresponding language command. For example, for the command *"fly through the gate"* and the timestamp `20250528_122154350309`, the resulting directory structure is shown in Figure 2.1.

To convert recorded trajectories into training-ready formats, we provide a script `data_2_embedding`, which processes the raw data and saves it as a tensor dataset in a `.pth` file. This conversion includes embedding both the language command and the images. The script supports two embedding configurations:

```
fly through the gate
├─ Images20250528_122154350309
│   ├─ 0.png
│   ├─ 1.png
│   └─ ...
└─ data20250528_122154350309
```

Figure 2.1: File structure for a recorded trajectory.

1. Use CLIP to embed both the language command and the images into 512-dimensional feature vectors each.
2. Use CLIP for the language command and ResNet18 for the image embeddings, both resulting in 512-dimensional vectors.

Since the language command remains constant for an entire trajectory, we store the text embedding only in the first entry of the dataset. The final `.pth` file has the following structure:

```
[
  {"text_embedding": 512-dim, "image_embedding": 512-dim,
   "state": 7-dim, "action": 4-dim},
  {"image_embedding": 512-dim, "state": 7-dim, "action": 4-dim},
  {"image_embedding": 512-dim, "state": 7-dim, "action": 4-dim},
  ...
]
```

This two-stage data collection and embedding pipeline offers several advantages. First, it allows fast experimentation with different deep learning models by modifying the embedding strategy, without the need to recollect data. Second, the pipeline supports data collection in both simulation and the real-world by changing the input topics of the image, state, and velocity command in the provided scripts. This adaptability makes it easy to extend the system to different hardware or environments.

2.2 Models

In this section, we describe the different neural network architectures used to learn control policies for autonomous drone navigation based on visual input, language commands, and the drone's state.

2.2.1 MLP

As a proof of concept, we began with a small MLP architecture consisting of five fully connected linear layers. The goal of this network is to map the input composed of state, language, and image embeddings to a 4-dimensional velocity command for the drone.

The drone's true action space is defined as $A_{\text{drone}} = [-1.5, 1.5]^4$. To improve training stability, we normalize this space by dividing each output component by 1.5, effectively transforming the action space to $A_{\text{drone}} = [-1, 1]^4$. Similarly, the 7-dimensional state vector is normalized so that each of its elements also lies within the interval $[-1, 1]$.

For activation functions, we use the ReLU nonlinearity after each hidden layer, and a `tanh` activation function in the final layer to ensure the output remains within the normalized action space. An overview of the MLP architecture and its training hyperparameters is provided in Table 2.1. The integration of the MLP with the input data format described in Section 2.1 is illustrated in Figure 2.2.

Component	**Details**
Image Embeddings	512-dim, CLIP
Language Embeddings	512-dim, CLIP
Drone State	7-dim
Input Dimension	1031, image + text + state
Hidden Layers	4
Hidden Layer Sizes	[2048, 1024, 512, 256]
Activation Function	ReLU (hidden), `tanh` (output)
Output Dimension	4, (v_x, v_y, v_z, yaw_rate)
Optimizer	Adam
Learning Rate	8e-4
Batch Size	1024
Epochs	50
Loss Function	MSE

Table 2.1: Architecture and training hyperparameters of the MLP model.

This setup gets evaluated both in simulation and in the real world in Chapter 3.

2.2.2 LangBeT

To improve our model, we explored the conditional BeT [8, 3]. This transformer architecture has previously been used in imitation learning tasks, such as learning to drive in a simulated environment by conditioning on front-facing camera images and high-level task descriptions like *"turn left"*, using the CARLA simulator [4].

BeT operates as follows: It takes an observation as input and outputs a continuous action. During training, illustrated in Figure 2.3, a k-means-based clustering is applied to the continuous action space to discretize it into bins. Each action is then represented by a discrete bin and a residual continuous offset from the bin center. This is referred to as the k-means encoder. The transformer, based on MinGPT, learns to predict:

- A categorical, discrete distribution over the bins, and
- A residual, continuous offset conditioned on the selected bin.

The discrete prediction is trained using a focal loss based on negative log-likelihood, while the residual component is trained with a multi-task loss. Dur-

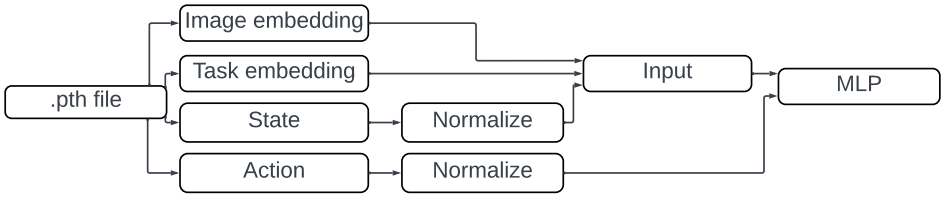


Figure 2.2: Connection between the stored `.pth` files, see Section 2.1, and the MLP

ing inference, see Figure 2.4, BeT first samples a bin and then adds the predicted offset to reconstruct the final continuous action.

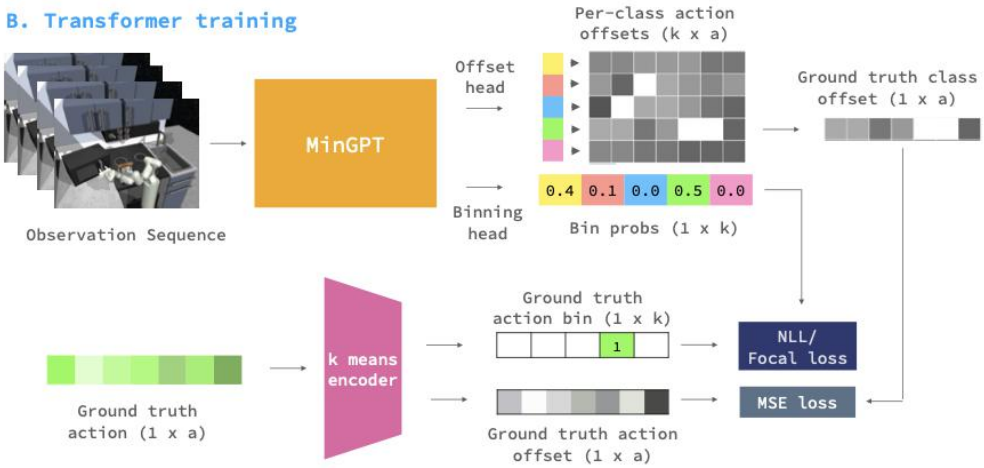


Figure 2.3: Training of BeT [8]

However, we used the conditional BeT, which takes an observation sequence and a goal sequence as input and outputs a continuous action. In our setup, the observation consisted of the last 5 image embeddings concatenated with the corresponding 5 drone states. For the goal sequence, we repeated the language embedding 5 times to match the temporal length of the observation input. The use of 5 time steps was intended to enable the model to capture temporal dependencies.

We adopted a transformer architecture with 12 layers and an embedding di-

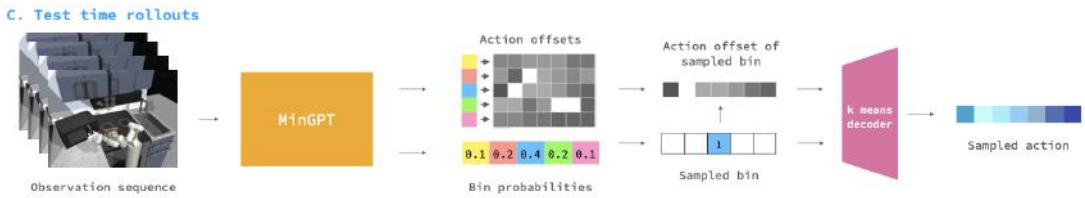


Figure 2.4: Rollout of BeT [8]

mension of 768, matching the configuration used in the CARLA experiments [4, 8, 3]. However, we later realized that we had misunderstood the intended usage of the goal sequence, which we elaborate on in Section 3.2.

2.2.3 Personal Transformer

Since LangBeT did not perform as expected, we implemented a transformer-based architecture from scratch. This model consists of three main components: an input embedding layer, a transformer encoder, and a final prediction head.

The input to the model is a sequence of 5 feature vectors, each of dimensionality 1031, resulting in an input shape of [5,1031]. Each feature vector is formed by concatenating:

- a 512-dimensional image embedding of the last 5 timesteps,
- a 512-dimensional language embedding, repeated across all 5 time steps,
- and a 7-dimensional state vector of the drone of the last 5 timesteps.

The input embedding layer projects each 1031-dimensional vector into a 1200-dimensional latent space. The embedded sequence is then processed by a transformer encoder with 3 layers and 4 attention heads per layer, using GELU activations and a feedforward dimension of 1200. From the encoded sequence, only the output of the last time step is selected, which is then passed through a final linear layer to produce a 4-dimensional action vector. An illustration of how the input sequence is transformed into control outputs by the Transformer model is shown in Figure **??**, with tensor dimensions annotated for clarity. Moreover, an overview of the model architecture and training hyperparameters is provided in Table 2.2.

This architecture allows the model to capture temporal dependencies across the last five observations, enabling it to generate context-aware action predictions based on recent history.

2.3 Overall Pipeline Overview

Figure 2.6 illustrates the full pipeline we implemented for language-conditioned drone control. Our approach supports multiple interchangeable components such as CLIP or ResNet18 for vision encoding, and MLP or Transformer for control policy learning. The modularity of the pipeline allows for flexible experimentation with different models and encoders while keeping the data format

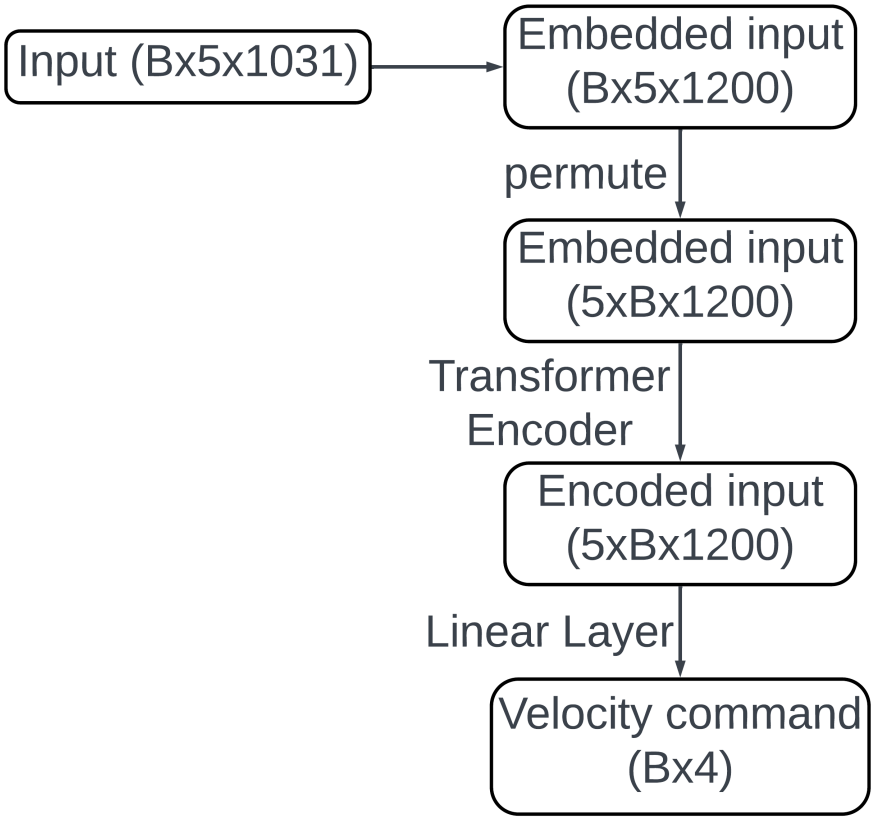


Figure 2.5: Transformer architecture with dimensional annotations. The batch size is represented by B.

and training procedure very similar.

Component	Details
Image Embeddings	512-dim, CLIP
Language Embeddings	512-dim, CLIP
Drone State	7-dim
Sequence Length	5
Input per Timestep	1031, image + text + state
Transformer Layers	3
Number of Heads	4
Feedforward Dimension	1200
Activation Function	GELU
Output Dimension	4 (v_x, v_y, v_z, yaw_rate)
Optimizer	Adam
Learning Rate	1e-4
Batch Size	64
Epochs	20
Loss Function	Huber

Table 2.2: Architecture and training hyperparameters of the Transformer model.

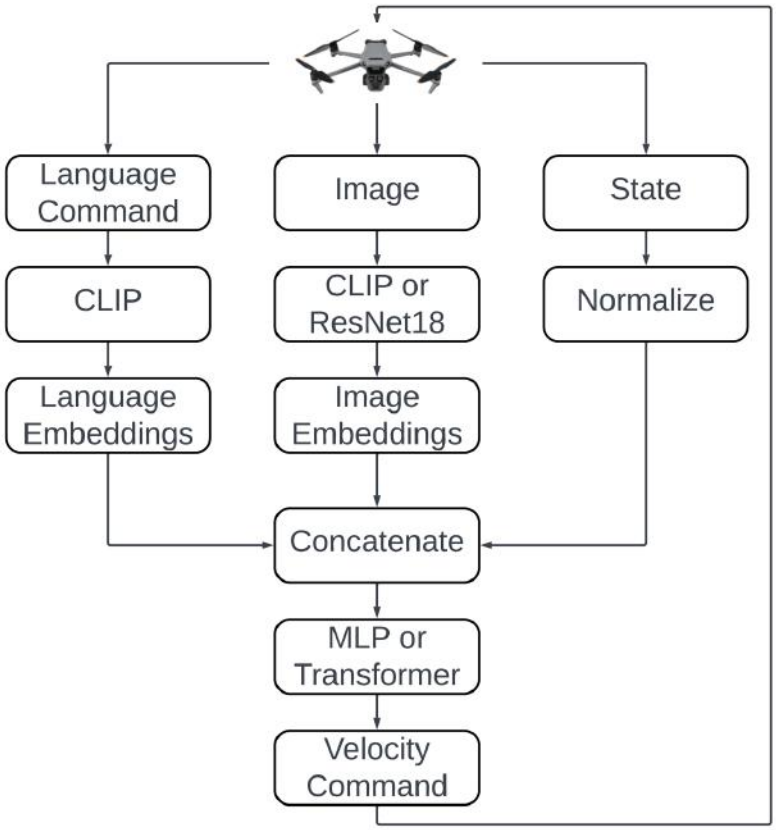


Figure 2.6: Overview of the end-to-end drone control pipeline. Language, image, and state inputs are embedded and concatenated to form a feature vector, which is then processed by either an MLP or Transformer to predict velocity commands.

Chapter 3

Experiments

In this chapter, we evaluate the performance of our trained models: the MLP, LangBeT, and a custom Transformer, in both simulated and real-world environments. Each model is assessed based on its ability to interpret natural language commands and navigate a drone accordingly using visual and state information. To this end, we collected two datasets: one in simulation consisting of 86 trajectories, and one in the real-world containing 237 trajectories. Each trajectory is associated with a specific language command such as *"fly through the gate"*, *"fly over the gate"*, *"fly under the gate"*, *"fly left around the gate"*, or *"fly right around the gate"*.

As a proof of concept, we limit our evaluation to a single scene in simulation, a spaceship environment, with one gate positioned in front of the drone, see Figure 3.1, and to a single room in the real-world setup, see Figure 3.2.

The primary goal of these experiments is to assess the robustness and generalization capabilities of the models when exposed to perturbations, such as:

- **Gate displacement**: Changes in the gate's position.
- **Language variation**: Slight errors or variations in the language command.

3.1 MLP

In this section, we evaluate the performance of our MLP model in both simulation and real-world settings.

3.1.1 Simulation

For our simulation experiments, we tested the model using a set of fixed language commands under various gate displacement conditions. Some commands were identical to those seen during training, while others were slight variations designed to assess the model's robustness to linguistic perturbations.

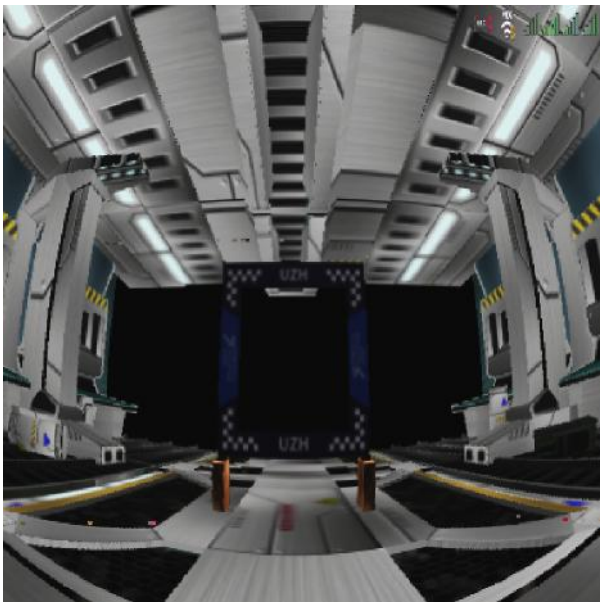


Figure 3.1: Spaceship simulation environment used for evaluation.

Because the disturbances in the simulation environment are little, we conducted each experiment only once per condition. A trajectory is considered successful if the drone executed the correct maneuver as intended by the language command.

In Table 3.1, we present the results. A displacement of +0.5m corresponds to a lateral shift of the gate 0.5 meters to the left of the drone's initial FoV. The **"In Dataset/Total Success Rate"** column indicates whether that exact command was present in the training data or the overall success rate over all language commands and gate displacements.

We observe that the model is able to successfully complete trajectories it was trained on, even when the gate is slightly displaced. This robustness is partly because the training dataset included some trajectories with minor gate displacements. Furthermore, the model can handle slight variations in language commands as long as the semantic meaning remains consistent which demonstrates that the CLIP text embeddings generalize reasonably well. Overall, we achieve a success rate of 72.22%, which is relatively high considering that some commands had a 0% success rate due to their significant deviation from the language commands seen during training.
However, when the semantics of a command differ significantly as for example for the *"ly through the gate"* command, the resulting embeddings are also significantly different. This leads to a failure in executing the correct trajectory, as the model can no longer associate the new command with the intended action.

In conclusion, the model performs well on commands seen during training and on semantically similar variants, but struggles when the language deviates too far in meaning from the training data.

Figure 3.2: Real-world environment used for evaluation

3.1.2 Real World

For our real-world experiments, we evaluated the MLP model described in Section 2.2.1 using a fixed set of language commands under various gate displacement conditions and slight variations in the commands themselves. Specifically, we tested the model with three different gate positions: +0.25m, 0m, and -0.25m. Here, +0.25m corresponds to a lateral shift of the gate to the left by 0.25m from the drone's point of view, and the 0m setup is shown in Figure 3.2. The goal of these experiments was to provide a proof of concept and to qualitatively assess the performance of CLIP's language and image embeddings in a real-world scenario. A trajectory was considered successful if the drone executed the behavior intended by the language command. Due to inevitable disturbances in real-world settings, each language command was tested three times. The results are summarized in Table 3.2, similar to the simulation setup in Section 3.1.1.

We observe that the model successfully executed some commands while struggling with others. Several key insights emerged:

First, it's important to note that the model was also trained using data in which the gate was positioned significantly lower than in the setup shown in Figure 3.2. This explains the poor performance for commands like *"fly through the gate"* and *"fly over the gate"*. In the former case, the drone consistently flew too low

Language Command	**Gate Displacement**			**In Dataset/ Total Success Rate**
	+0.5m	**0m**	**-0.5m**	
fly through the gate	✓	✓	✓	✓
Fly through the gate	✓	✓	✓	✓
fly over the gate	✓	✓	✗	✓
fly under the gate	✓	✓	✓	✓
fly left around the gate	✓	✓	✓	✓
fly right around the gate	✓	✓	✓	✓
fly through gate	✓	✓	✓	✓
Fly through gate	✓	✓	✓	✗
fly through teh gate	✓	✓	✓	✗
fly through	✗	✗	✗	✗
fy through the gate	✗	✗	✗	✗
ly through the gate	✗	✗	✗	✗
Success Rate	**75%**	**75%**	**66.67%**	**72.22%**

Table 3.1: Performance of the MLP model under different gate displacement conditions using various language commands. The image embeddings were generated using CLIP.

and collided with the bottom of the gate; in the latter, it always flew through the gate instead of over it.

Second, the model clearly responds differently to semantically different language commands. Moreover, commands that are semantically similar but contain minor typographical errors such as "fly undr teh gate" were interpreted correctly. This again demonstrates the robustness of CLIP's language embeddings to small variations in phrasing and also shows that it can differentiate between semantic different language commands.

However, the model had significant difficulty conditioning on image embeddings. For instance, in the *"fly undr gate"* command, crashes consistently occurred into the left post of the gate because the model failed to recognize that the gate had been shifted to the right. A similar issue occurred with *"fly right around the gate"*, where the drone collided because it failed to adjust for the new gate position. Moreover, the drone consistently flew slightly too low, suggesting that the model could not account for the change in gate height. This further indicates weak conditioning on the visual input.

In conclusion, CLIP's language embeddings appear to be reliable in a real-world context, handling command variations effectively. In contrast, the model shows limitations in utilizing CLIP's image embeddings, likely mostly due to the subtle nature of visual changes in the environment (e.g., minor shifts in gate position) compared to the more distinct variations in language input. This highlights a key challenge in real-world multimodal learning: conditioning on subtle visual cues is inherently more difficult than interpreting varied language inputs. However, despite those challenges, our small MLP model was able to achieve an overall success rate of 62.22% in a real-world setup.

Language Command	**Gate Displacement**									**In Dataset/ Total Success Rate**
	+0.25m			**0m**			**-0.25m**			
fly through the gate	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
fly over the gate	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
fly under the gate	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
fly left around the gate	✓	✗	✗	✓	✗	✗	✓	✗	✗	✓
fly right around the gate	✓	✓	✓	✓	✓	✓	✗	✓	✗	✓
fly left around gate	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓
fly right around gate	✓	✓	✓	✗	✗	✗	✗	✗	✗	✓
fly under gate	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
fly undr gate	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗
fly undr teh gate	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗
Success Rate	**70%**			**63.33%**			**53.33%**			**62.22%**

Table 3.2: Performance of the MLP model in the real world under different gate displacements and various language commands.

3.2 LangBeT

In this section, we evaluate the training performance of our LangBeT model under various configurations and clarify a key misunderstanding related to the use of the goal sequence.

We first trained several conditional BeT models using CLIP image embeddings concatenated with drone state information as the observation sequence, and CLIP-generated language command embeddings as the goal sequence, as described in Section 2.2.2. All models were trained for 160 epochs, and we experimented with different transformer depths (6, 12, and 24 layers) to assess their impact. The resulting training and evaluation losses are shown in Figure 3.3, trained on the real-world dataset. Despite the expected decrease in training loss, the evaluation loss never went further down than 4.8 for all layer configurations, indicating poor generalization.

To investigate whether the inclusion of state information was affecting performance, we repeated the experiment without incorporating state data into the observation sequence. The corresponding results are shown in Figure 3.4. However, the evaluation loss again failed to drop below 4.7, suggesting that the state inputs were not the root cause of the problem. Next, we repeated the experiment using ResNet18 embeddings instead of CLIP for image representation. However, the results were again similar, which ruled out CLIP embeddings as the source of the issue.

Finally, we identified the core problem: a misunderstanding in how to use the goal sequence in conditional BeT. Initially, we interpreted the goal sequence as a static task description. However, the intended use is to provide an observation sequence that reflects what the final goal state for that demonstration should look like (e.g., an image embedding concatenated with the state of the drone of the goal location for that demonstration).

To address this, we attempted to train the conditional BeT model without a goal sequence, effectively using it in an unconditioned setting. However, this approach did not lead to successful training. We also experimented with using the standard, unconditioned BeT architecture in our setup, but were similarly unsuccessful.

3.3 Personal Transformer

In this section, we evaluate the performance of our Transformer model in a simulated environment.

For the simulation experiment, we tested the Transformer using the same set of language commands and gate displacement conditions as in the MLP experiment, see Section 3.1.1. However, because transformer-based models typically require more training data than MLPs, we collected an additional 47 demonstration trajectories. This resulted in a total of 133 demonstrations for training the Transformer model. As before, some of the language commands used during testing were identical to those seen in training, while others were slightly varied to assess the model's robustness to linguistic perturbations.

Again, given the minimal disturbances in the simulation environment, we performed only one trial per condition. A trajectory was considered successful if the drone executed the correct maneuver corresponding to the language command. Table 3.3 presents the results of our experiment. As in Section 3.1.1, a displacement of +0.5 meters indicates that the gate was shifted 0.5 meters to the left of the drone's initial FoV.

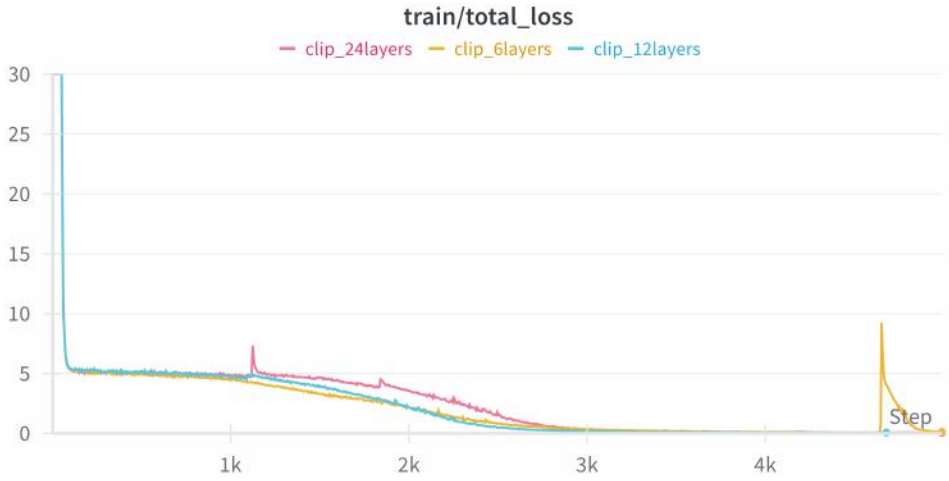
Language Command	**Gate Displacement**			**In Dataset/ Total Success Rate**
	+0.5m	**0m**	**-0.5m**	
fly through the gate	✓	✓	✓	✓
Fly through the gate	✓	✓	✓	✓
fly over the gate	✓	✓	✗	✓
fly under the gate	✓	✓	✗	✓
fly left around the gate	✓	✓	✓	✓
fly right around the gate	✓	✓	✓	✓
fly through gate	✓	✓	✓	✓
Fly through gate	✓	✓	✓	✗
fly through teh gate	✓	✓	✓	✗
fly through	✗	✗	✗	✗
fy through the gate	✓	✓	✗	✗
ly through the gate	✗	✗	✗	✗
Success Rate	**83.33%**	**83.33%**	**58.33%**	**75%**

Table 3.3: Performance of the Transformer model under different gate displacement conditions using various language commands. The image embeddings were generated using CLIP.

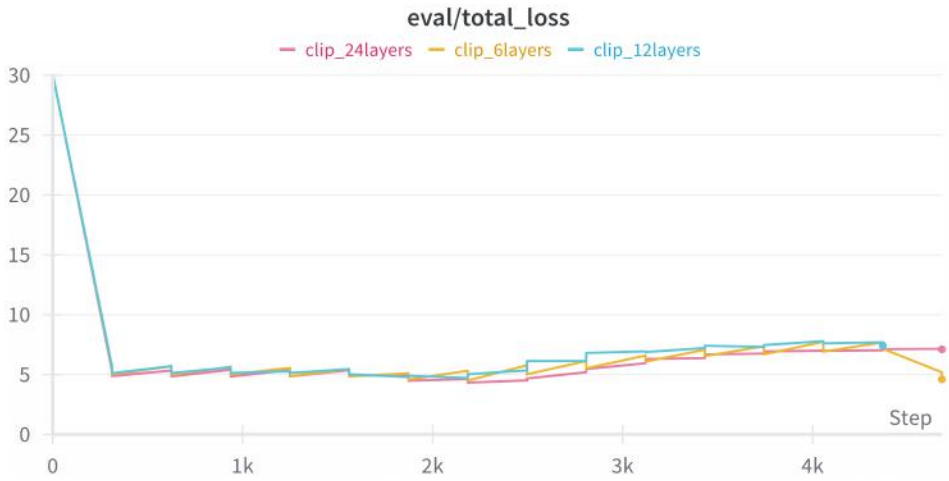
Similar to the MLP model, the Transformer performs well in simulation, successfully executing most language commands under various gate displacement conditions. The model also handles slight variations in the language input that

do not change the semantic meaning—for instance, *"fly through gate"* instead of *"fly through the gate"*. This again highlights the effectiveness of CLIP language embeddings, as they appear to preserve semantic similarity across minor linguistic changes. Furthermore, the results indicate that the Transformer responds appropriately to semantically different commands. For example, the drone's behavior changes significantly when given commands like *"fly under the gate"* versus *"fly through the gate"*, demonstrating that the language embeddings change significantly in those cases. However, consistent with the MLP results, CLIP image embeddings appear less reliable. Since the only visual variation is the gate's lateral position, the embeddings may not capture this subtle change robustly, leading to performance limitations.

In contrast to the MLP, the Transformer offers greater potential for scaling to more complex tasks and for generalizing across larger, more diverse datasets. Furthermore, its ability to capture temporal dependencies makes it particularly well-suited for tasks involving sequential decision-making, which is essential in most real-world applications. Even in this relatively trivial simulation setup, the Transformer outperforms the MLP, achieving an overall success rate of 75% compared to 72.22% for the MLP.

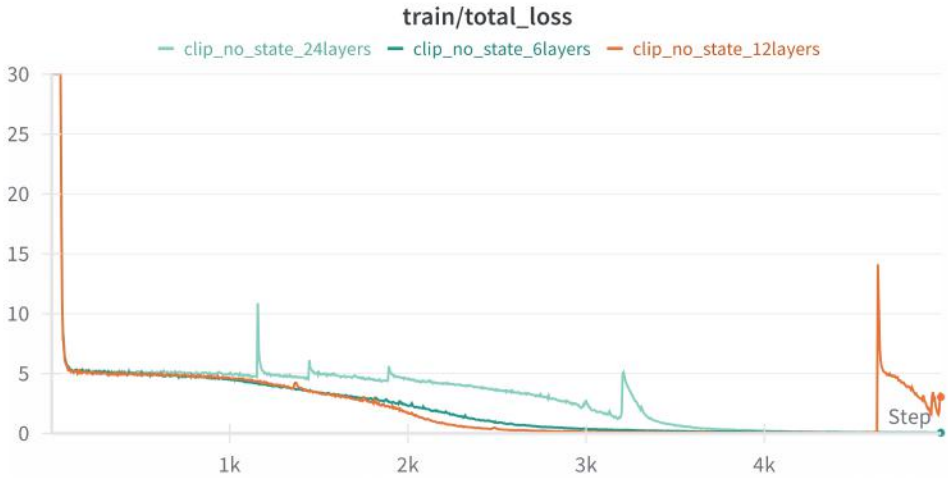


(a) Training loss for different transformer depths

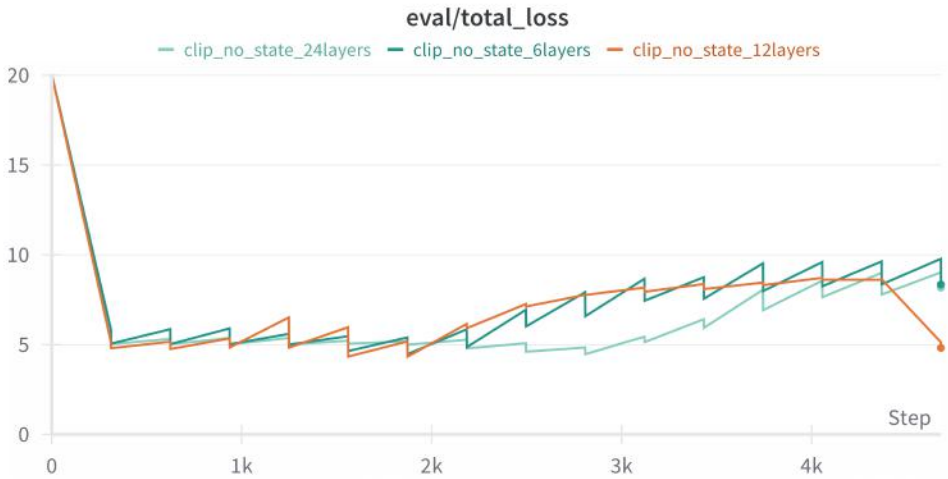


(b) Evaluation loss for different transformer depths

Figure 3.3: Training and evaluation losses using CLIP embeddings with state information in the observation sequence



(a) Training loss for different transformer depths



(b) Evaluation loss for different transformer depths

Figure 3.4: Training and evaluation losses using CLIP embeddings without state information in the observation sequence

Chapter 4

Discussion

One of the key strengths of this project lies in the flexibility and modularity of the data collection pipeline. It can be easily adapted to different experimental setups by specifying the target folder for data storage and the relevant ROS topics for the image stream, drone state, and velocity commands. Once data is collected, we provide a script that converts the raw inputs into embeddings and stores them in .pth files, ready for model training. The script is designed for extensibility: users can easily plug in their own feature extractors, and we provide implementations for both CLIP and ResNet18. The embedding model can be selected via a command-line parameter. The primary limitation of this pipeline is that it requires a ROS environment, as data must be published to ROS topics.

The MLP model served as a useful baseline to validate our pipeline and design assumptions. It demonstrated that our approach is functional and that meaningful learning can occur, even with a relatively small model. However, its generalization capacity is limited: it performs well on a small, fixed set of tasks but struggles with more diverse or unseen conditions.

Although the LangBeT model did not yield successful results, it was instrumental in shaping our understanding of how to structure the dataset and prepare input sequences for transformer-based models. This foundation enabled the development of our final model, a Personal Transformer architecture. The main advantage of this transformer was its ability to condition on past observations, allowing it to model temporal dependencies. Nonetheless, it still struggled to reliably detect subtle spatial changes in the environment, such as small shifts in gate position.

4.1 Conclusion

In this thesis, we developed and validated a complete pipeline for language-conditioned drone control, from data collection to model training and evaluation. The data collection system proved to be both flexible and extensible, making it applicable to a wide range of robotic scenarios. We also implemented and tested

two different models: an MLP and a Personal Transformer. The MLP demonstrated the feasibility of our setup, while the Transformer introduced temporal reasoning into the system. Despite the challenges faced, particularly with the more complex LangBeT architecture, our final Transformer model showed that the pipeline can handle sequential data and partially adapt to linguistic variations and environmental perturbations. This work lays a solid foundation for future research in language-guided robotics.

4.2 Future Work

Future work could proceed in several promising directions. A primary goal would be to fully integrate a more sophisticated transformer-based model such as conditional BeT [8, 3] into our pipeline. Properly leveraging the goal-conditioning mechanism of BeT could significantly improve model performance, especially in tasks involving complex language instructions. Another avenue would be to experiment with different models for generating image and language embeddings. For instance, using more recent vision-language models or fine-tuned encoders could yield better perceptual representations tailored to the drone's tasks. Additionally, evaluating the system in more varied and dynamic environments would help assess its robustness and generalization capabilities.

Bibliography

[1] Hengxing Cai, Jinhan Dong, Jingjun Tan, Jingcheng Deng, Sihang Li, Zhifeng Gao, Haidong Wang, Zicheng Su, Agachai Sumalee, and Renxin Zhong. Flightgpt: Towards generalizable and interpretable uav vision-and-language navigation with vision-language models, 2025.

[2] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Bıyık, Hongxu Yin, Sifei Liu, and Xiaolong Wang. Navila: Legged robot vision-language-action model for navigation, 2025.

[3] Zichen Jeff Cui, Yibin Wang, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. From play to policy: Conditional behavior generation from uncurated robot data. *arXiv preprint arXiv:2210.10047*, 2022.

[4] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator, 2017.

[5] Noriaki Hirose, Catherine Glossop, Ajay Sridhar, Dhruv Shah, Oier Mees, and Sergey Levine. Lelan: Learning a language-conditioned navigation policy from in-the-wild video. In *Conference on Robot Learning*, 2024.

[6] Jack Saunders, Sajad Saeedi, and Wenbin Li. Autonomous aerial robotics for package delivery: A technical review. *Journal of Field Robotics*, 41(1):3–49, July 2023.

[7] Pranav Saxena, Nishant Raghuvanshi, and Neena Goveas. Uav-vln: End-to-end vision language guided navigation for uavs, 04 2025.

[8] Nur Muhammad Mahi Shafiullah, Zichen Jeff Cui, Ariuntuya Altanzaya, and Lerrel Pinto. Behavior transformers: Cloning k modes with one stone. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.

[9] Hazim Shakhatreh, Ahmad H. Sawalmeh, Ala Al-Fuqaha, Zuochao Dou, Eyad Almaita, Issa Khalil, Noor Shamsiah Othman, Abdallah Khreishah, and Mohsen Guizani. Unmanned aerial vehicles (uavs): A survey on civil applications and key research challenges. *IEEE Access*, 7:48572–48634, 2019.

[10] Yunlong Song, Selim Naji, Elia Kaufmann, Antonio Loquercio, and Davide Scaramuzza. Flightmare: A flexible quadrotor simulator, 2021.

ROBOTICS &
PERCEPTION
GROUP

Title of work:

Language-Guided Drone Control

Thesis type and date:

Semester Thesis, June 2025

Supervision:

Ismail Geles
Ángel Romero
Dr. Elie Aljalbout
Prof. Dr. Davide Scaramuzza

Student:

Name:	Laurent Schroeder
E-mail:	lschroeder@student.ethz.ch
Legi-Nr.:	21-949-201

Statement regarding plagiarism:

By signing this statement, I affirm that I have read the information notice on plagiarism, independently produced this paper, and adhered to the general practice of source citation in this subject-area.

Information notice on plagiarism:

http://www.lehre.uzh.ch/plagiate/20110314_LK_Plagiarism.pdf

Zurich, 2. 6. 2025: _____